DE Challenge 1 — ETL Basics with Python and SQL

Overview

End-to-end ETL pipeline built in Python that extracts NYC Yellow Taxi trip data, applies data quality and enrichment transformations, loads the results into a SQLite star-schema database, and answers analytical questions with SQL queries.

Dataset

Field	Detail
Name	NYC TLC Yellow Taxi Trip Records
Source	NYC Taxi & Limousine Commission (TLC)
File used	`yellow_tripdata_2023-01.parquet` (January 2023)
Format	Apache Parquet
License	Public domain — NYC Open Data. No restrictions on use, distribution, or modification.
Direct URL	`https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2023-01.parquet`

Original Columns

Column	Type	Description
`VendorID`	int	TPEP provider (1 = Creative Mobile, 2 = VeriFone)
`tpep_pickup_datetime`	datetime	Timestamp when the meter was engaged
`tpep_dropoff_datetime`	datetime	Timestamp when the meter was disengaged

`passenger_count`	float	Number of passengers
`trip_distance`	float	Trip distance in miles
`RatecodeID`	float	Final rate code (1–6)
`store_and_fwd_flag`	str	Whether trip was held in memory before sending
`PULocationID`	int	TLC Taxi Zone for pickup
`DOLocationID`	int	TLC Taxi Zone for dropoff
`payment_type`	float	Payment method code (1–6)
`fare_amount`	float	Meter fare
`extra`	float	Extras and surcharges
`mta_tax`	float	MTA tax
`tip_amount`	float	Tip amount
`tolls_amount`	float	Tolls paid
`improvement_surcharge`	float	$0.30 improvement surcharge
`total_amount`	float	Total charged to passengers
`congestion_surcharge`	float	NYC congestion surcharge
`airport_fee`	float	Airport pickup fee

Pipeline Structure

NYC_Taxi_ETL.ipynb

Transformations

A. Column Renaming

Standardized column names to `snake_case` for SQL and Python consistency.

Original	Renamed
`VendorID`	`vendor_id`
`tpep_pickup_datetime`	`pickup_datetime`
`tpep_dropoff_datetime`	`dropoff_datetime`
`RatecodeID`	`rate_code_id`
`PULocationID`	`pickup_location_id`
`DOLocationID`	`dropoff_location_id`

B. Type Casting

Enforced correct data types to prevent downstream errors.

Column	Cast to	Reason
`pickup_datetime`	`datetime64`	Enable time arithmetic
`dropoff_datetime`	`datetime64`	Enable time arithmetic
`vendor_id`	`Int64`	Nullable integer (avoids float)
`passenger_count`	`Int64`	Nullable integer
`payment_type`	`Int64`	Nullable integer
`rate_code_id`	`Int64`	Nullable integer

C. Data Cleansing

Removed logically invalid records to ensure data quality.

Rule	Rationale
`trip_distance > 0`	A trip with zero distance is invalid
`fare_amount > 0`	Zero or negative fares are likely data errors
`passenger_count > 0`	Trips must have at least one passenger
`pickup_datetime < dropoff_datetime`	Dropoff must occur after pickup

D. Deduplication

Removed exact duplicate rows based on the combination of columns that uniquely identifies a trip.

Deduplication key: `pickup_datetime`, `dropoff_datetime`, `pickup_location_id`, `dropoff_location_id`, `fare_amount`

E. Derived Columns

Enriched the dataset with computed features for analysis.

New Column	Formula	Description
`trip_duration_min`	`(dropoff - pickup).seconds / 60`	Trip duration in minutes
`speed_mph`	`trip_distance / (duration / 60)`	Average speed in mph
`total_per_mile`	`total_amount / trip_distance`	Cost efficiency per mile
`pickup_hour`	`pickup_datetime.dt.hour`	Hour of day (0–23)
`pickup_date`	`pickup_datetime.dt.date`	Calendar date of pickup

F. Aggregation

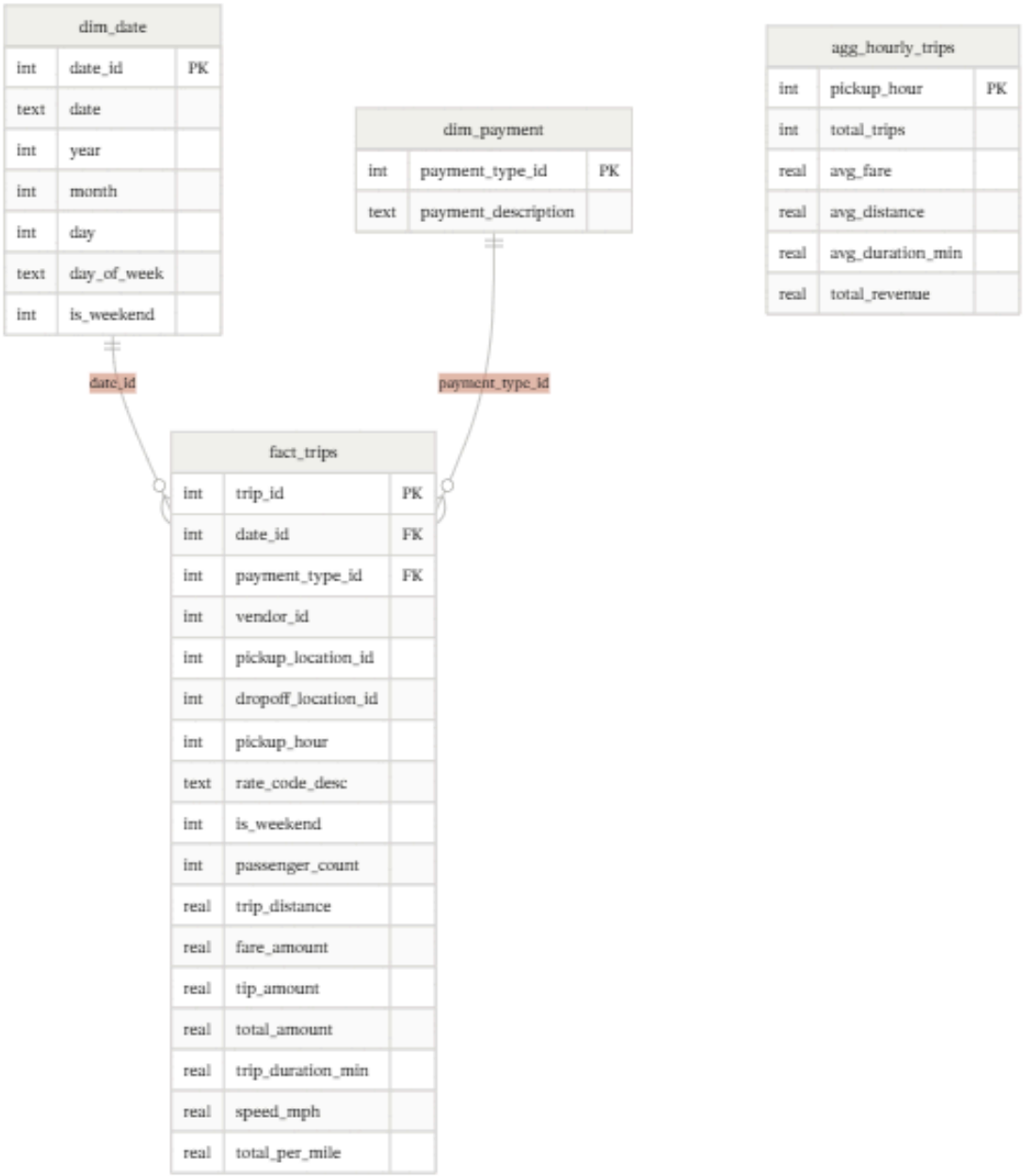
Pre-aggregated trip statistics by pickup hour for fast analytical access.

Metric	Description
`total_trips`	Count of trips per hour

`avg_fare`	Mean fare amount
`avg_distance`	Mean trip distance in miles
`avg_duration_min`	Mean trip duration in minutes
`total_revenue`	Sum of total_amount per hour

Star Schema

The cleaned data is loaded into a **SQLite** database (`nyc_taxi.db`) following a star schema design.



Tables

`dim_date` — Calendar dimension

Column	Type	Description
`date_id`	INTEGER PK	Surrogate key
`date`	TEXT	Date string (YYYY-MM-DD)
`year`	INTEGER	Calendar year
`month`	INTEGER	Calendar month (1–12)
`day`	INTEGER	Day of month
`day_of_week`	TEXT	Day name (Monday…Sunday)
`is_weekend`	INTEGER	1 if Saturday or Sunday

`dim_payment` — Payment method dimension

Column	Type	Description
`payment_type_id`	INTEGER PK	TLC payment code
`payment_description`	TEXT	Human-readable label

Payment codes: 1 Credit Card · 2 Cash · 3 No Charge · 4 Dispute · 5 Unknown · 6 Voided Trip

`fact_trips` — Central fact table

Column	Type	Description
`trip_id`	INTEGER PK	Auto-increment surrogate key
`date_id`	INTEGER FK	References `dim_date`
`payment_type_id`	INTEGER FK	References `dim_payment`
`vendor_id`	INTEGER	TPEP vendor
`pickup_location_id`	INTEGER	TLC zone for pickup

`dropoff_location_id`	INTEGER	TLC zone for dropoff
`pickup_hour`	INTEGER	Hour of pickup (0–23)
`rate_code_desc`	TEXT	Rate description
`is_weekend`	INTEGER	1 if weekend trip
`passenger_count`	INTEGER	Number of passengers
`trip_distance`	REAL	Distance in miles
`fare_amount`	REAL	Base meter fare
`tip_amount`	REAL	Tip paid
`total_amount`	REAL	Total charged
`trip_duration_min`	REAL	Duration in minutes
`speed_mph`	REAL	Average speed
`total_per_mile`	REAL	Cost per mile

`agg_hourly_trips` — Pre-aggregated summary

Pre-computed hourly statistics used to answer analytical queries without scanning the full fact table.

Analytical Queries

Three SQL queries are included in the notebook to demonstrate the star schema in use.

Q1 — Peak Hours by Trip Volume

Identifies the 10 busiest hours of the day, ranked by number of trips. Useful for demand forecasting and driver scheduling.

```sql
SELECT
    pickup_hour,
    COUNT(trip_id)                AS total_trips,
    ROUND(AVG(trip_duration_min), 2)  AS avg_duration_min,
    ROUND(AVG(trip_distance), 2)      AS avg_distance_miles,
    ROUND(SUM(total_amount), 2)       AS total_revenue
FROM fact_trips
GROUP BY pickup_hour
ORDER BY total_trips DESC
LIMIT 10;
```

Q2 — Top 10 Pickup Locations

Ranks TLC taxi zones by number of pickups and includes average fare and distance. Useful for identifying high-demand zones.

```sql
SELECT
    pickup_location_id,
    COUNT(trip_id)              AS total_pickups,
    ROUND(AVG(fare_amount), 2)    AS avg_fare,
    ROUND(AVG(trip_distance), 2)  AS avg_distance_miles
FROM fact_trips
GROUP BY pickup_location_id
ORDER BY total_pickups DESC
LIMIT 10;
```

Q3 — Weekday vs Weekend Comparison

Compares trip behavior across weekdays and weekends using the date dimension join. Shows differences in volume, distance, speed, and revenue.

```sql
SELECT
    CASE WHEN d.is_weekend = 1 THEN 'Weekend' ELSE 'Weekday' END AS period,
    COUNT(f.trip_id)                  AS total_trips,
    ROUND(AVG(f.trip_distance), 2)      AS avg_distance_miles,
    ROUND(AVG(f.trip_duration_min), 2)  AS avg_duration_min,
    ROUND(AVG(f.fare_amount), 2)        AS avg_fare,
    ROUND(AVG(f.speed_mph), 2)          AS avg_speed_mph,
    ROUND(SUM(f.total_amount), 2)       AS total_revenue
FROM fact_trips f
JOIN dim_date d ON f.date_id = d.date_id
GROUP BY d.is_weekend;
```

Requirements

pandas
numpy
sqlite3 (standard library)
matplotlib
pyarrow (for reading .parquet)

How to Run

1. Open `NYC_Taxi_ETL.ipynb` in Google Colab, Jupyter or any compatible environment.
2. Run all cells in order (Extract → Transform → Load → Queries).
3. The pipeline will download the dataset automatically on first run (~50 MB).
4. Output database: `nyc_taxi.db` (SQLite, created in the working directory).

LLM Prompts Used

The following prompts were used with an AI code assistant (Claude) for the writing of this document:

1. *"Write a .docx file documenting the dataset origin, column descriptions, all transformations, the star schema design, and the analytical queries."*